

Nom du Projet : "The Over-Engineered Guestbook"

(Le Livre d'Or Sur-Ingénieré)

Le but : Créer une application web simple (un livre d'or où les gens laissent un message), mais construite comme une architecture micro-services robuste de niveau production.

L'architecture cible (Ce que tu vas construire) : Tu vas avoir 5 conteneurs qui tournent et discutent ensemble :

1. **Frontend/Proxy :** Un serveur **Nginx** (reçoit les requêtes HTTP).
 2. **Backend API :** Une petite app (Python Flask ou Node.js, à toi de choisir) qui traite la logique.
 3. **Cache :** Une base **Redis** (pour compter le nombre de vues de la page rapidement).
 4. **Database :** Une base **PostgreSQL** (pour stocker les messages du livre d'or durablement).
 5. **Admin DB :** Un outil **Adminer** (interface graphique web pour voir ta base de données sans ligne de commande).
-



Jour 1 : La Base (Dockerfile & Réseau simple)

Objectifs :

1. **Coder l'API (Backend) :** Écris un script simple (ex: `app.py`) qui retourne "Hello World" sur une page web.
2. **Conteneuriser l'API :**
 - Crée un `Dockerfile` pour ton application.
 - Il doit copier ton code, installer les dépendances (ex: `pip install flask`) et lancer l'app.
 - **Contrainte :** Utilise une image de base légère (ex: `python:3.9-alpine` ou `slim`).
3. **Le Cache (Redis) :**
 - Lance un conteneur Redis officiel (via `docker run`).
 - Modifie ton code pour qu'à chaque fois qu'on rafraîchit la page, il incrémente un compteur dans Redis et l'affiche ("Page vue X fois").
 - Fais communiquer ton conteneur API et ton conteneur Redis (Indice : il faudra créer un `docker network`).

Compétence acquise : Création d'image, `docker build`, gestion manuelle du réseau (`docker network create`).



Jour 2 : L'Orchestration & La Persistance (Docker Compose)

Finis de lancer les commandes à la main. On passe aux choses sérieuses.

Objectifs :

1. **Le fichier magique** : Crée un fichier `docker-compose.yml`.
 - Il doit lancer l'API et Redis ensemble.
 - Plus besoin de lancer les conteneurs manuellement. Une seule commande `docker-compose up` doit tout allumer.
2. **Ajout de la Database (PostgreSQL)** :
 - Ajoute un service `postgres` dans ton `docker-compose`.
 - Modifie ton code API pour qu'il puisse :
 - Recevoir un message (POST).
 - Lister les messages (GET).
 - Sauvegarder tout ça dans Postgres.
3. **Le Piège (VOLUMES)** :
 - C'est le test crucial. Écris un message dans ta base.
 - Éteins tout (`docker-compose down`).
 - Supprime le conteneur.
 - Rallume tout.
 - **Tes données doivent être encore là**. Si la base est vide, tu as échoué. Tu dois configurer un **Volume Docker**.
4. **L'outil de debug** : Ajoute le service `adminer` dans le compose (port 8080) pour vérifier visuellement que tes données sont bien dans Postgres.

Compétence acquise : Infrastructure as Code, Variables d'environnement, Volumes (Persistance).



Jour 3 : "Production Ready" (Proxy & Optimisation)

On rend ça propre, comme en entreprise.

Objectifs :

1. **Le Reverse Proxy (Nginx)** :
 - On ne doit plus accéder directement à ton API Python/Node.
 - Ajoute un conteneur Nginx devant.
 - Configure Nginx (via un fichier `nginx.conf` que tu injectes dans le conteneur) pour qu'il redirige le trafic entrant vers ton API.
2. **Isolation Réseau (Sécurité)** :
 - Configure les réseaux dans le Compose.
 - Le monde extérieur accède à Nginx.
 - Nginx parle à l'API.
 - L'API parle à Redis et Postgres.
 - **INTERDIT** : Nginx ne doit pas pouvoir parler à la Base de données directement. La base de données ne doit pas être accessible depuis internet (pas de port mapping, sauf pour Adminer en dev).

3. Optimisation (Multi-stage build) :

- Reprends ton **Dockerfile** de l'API.
 - Utilise le "Multi-stage build" (étape de build vs étape de run) pour réduire la taille finale de ton image au maximum. Essaie d'être sous les 100Mo ou 200Mo selon le langage.
-

Le "Boss Final" (Critères de réussite)

Pour considérer le projet fini, tu dois pouvoir faire ceci :

1. Cloner ton dossier de projet sur un **nouvel ordinateur** (ou supprimer toutes tes images/conteneurs actuels).
2. Taper **UNE SEULE** commande : `docker-compose up -d --build`.
3. Tout doit s'installer, se lancer, et être accessible sur `localhost`.
4. Si je redémarre l'ordi, je ne perds pas mes messages.

Les mots-clés à chercher sur Google pour t'aider :

- *Docker Compose networking internal vs external*
- *Docker named volumes vs bind mounts*
- *Docker multi-stage build python/node*
- *Nginx reverse proxy docker config*
- *Environment variables docker compose postgres*

C'est un projet très formateur. Si tu arrives au bout du Jour 3 et que ça marche, tu as un niveau suffisant pour répondre à 80% des questions techniques sur Docker en entretien Junior/Confirmé.